

Atividade 2 - Arquitetura de Software + Katas

Nome: Arthur Feitosa Gonçalves Lima

Matrícula: 232000721

Projeto: Não há negócio como o e-business

O cliente deseja desenvolver uma plataforma SaaS multi-tenant que forneça ferramentas e infraestrutura para pequenas empresas criarem e operarem seus próprios portais de e-commerce. Cada empresa utiliza a mesma plataforma, mas seus dados, usuários, produtos, pedidos e configurações permanecem isolados logicamente.

Usuários e escala

A solução deve suportar até milhões de usuários por cliente e milhares ou mais de clientes, mantendo disponibilidade, desempenho e segurança mesmo em períodos de grande demanda.

Requisitos principais

Requisito	Descrição
Confiabilidade e escala	Alta disponibilidade, recuperação de falhas, balanceamento de carga e escalabilidade horizontal.
Gerenciamento de usuários	Cadastro, autenticação, perfis, papéis, permissões e isolamento entre clientes (tenants).
Pagamentos	Integração com gateways e diferentes meios de pagamento, com segurança e tratamento de transações.
Administração mínima	O cliente administra sua loja, catálogo, pedidos e configurações; a plataforma cuida da infraestrutura e operação.
Internacionalização obrigatória	Idiomas, moedas, formatos de data/número e regras tributárias/fiscais por região.
Integrações	APIs para transportadoras, antifraude, ERP/CRM, marketplaces, comunicação e serviços fiscais.

KATA — resumo do projeto

Problema: pequenas empresas precisam vender pela internet sem ter que administrar uma infraestrutura complexa e sem perder desempenho quando o número de usuários cresce.

Solução: plataforma SaaS multi-tenant para criação e operação de lojas virtuais, oferecendo catálogo, busca, carrinho, checkout, usuários, pedidos, pagamentos, estoque, frete, promoções, notificações, relatórios e integrações externas.

Escala: arquitetura preparada para milhares de clientes e milhões de usuários por loja, utilizando cache, filas/eventos, serviços independentes, balanceamento de carga e autoscaling.

Princípios: separação em camadas, microsserviços independentes, comunicação síncrona e assíncrona, multi-tenancy, segurança, observabilidade e infraestrutura em nuvem.

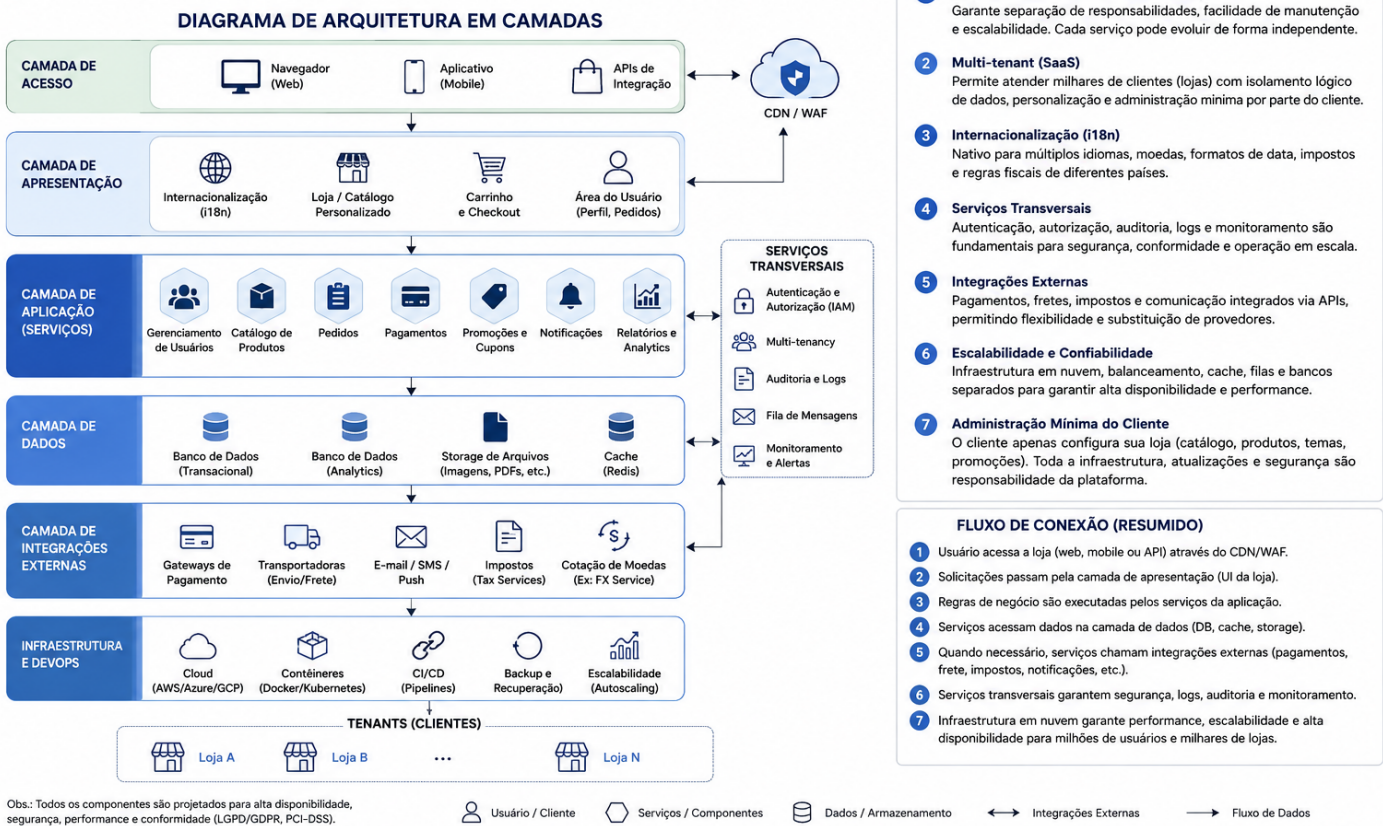
Resultado esperado: permitir que o cliente foque no negócio e nas vendas, enquanto a plataforma assume a complexidade técnica, segurança, disponibilidade, atualizações e escalabilidade.

DIAGRAMA DE ARQUITETURA EM CAMADAS

Diagrama detalhado do projeto, incorporado ao documento conforme a arquitetura definida para a plataforma de e-commerce.

KATA: Não há negócio como o e-business

Objetivo: Plataforma multi-tenant (SaaS) para criação de portais de e-commerce para pequenas empresas venderem aos seus clientes.



Justificativa das escolhas arquiteturais

Escolha	Por quê?
Arquitetura em camadas	Separa responsabilidades e facilita manutenção, testes, evolução e entendimento do sistema.
SaaS multi-tenant	Permite atender milhares de empresas com uma plataforma única, reduzindo custo operacional e evitando que cada cliente administre sua própria infraestrutura.
Microserviços	Permitem escalar funções críticas de forma independente, como catálogo, pedidos e pagamentos, sem aumentar todos os componentes ao mesmo tempo.
API Gateway	Centraliza entrada, autenticação, roteamento, limites de uso e versionamento das APIs.
Mensageria/eventos	Desacopla processos demorados e absorve picos de tráfego, melhorando resiliência e escalabilidade.
Cache + bancos especializados	Redis reduz latência; banco relacional preserva transações; busca e analytics usam estruturas adequadas a seus próprios tipos de consulta.
Cloud + autoscaling	Atende variações de demanda e a escala de milhões de usuários, com alta disponibilidade e recuperação de falhas.
Internacionalização	Trata idiomas, moedas, formatos e regras regionais desde a base da plataforma, pois é um requisito obrigatório.

Como as camadas se conectam

1. O usuário acessa a loja por navegador, aplicativo ou API. 2. CDN/WAF protege e distribui o tráfego. 3. A apresentação envia requisições ao API Gateway. 4. O Gateway encaminha a requisição ao microserviço responsável, considerando o tenant. 5. O serviço consulta cache/banco ou publica eventos em filas. 6. Quando necessário, integra-se com pagamento, frete, fiscal, antifraude e demais parceiros. 7. Logs, métricas e auditoria acompanham todo o fluxo.

Tecnologias sugeridas: React/Next.js; Flutter/React Native; Java/Spring Boot, .NET, Node.js ou Go; Kong/Apigee/AWS API Gateway; Kafka/RabbitMQ/SQS/SNS; PostgreSQL, MongoDB, Redis e OpenSearch; AWS/Azure/GCP; Docker/Kubernetes; Terraform; GitHub Actions/GitLab CI; Prometheus, Grafana, ELK e Jaeger.